

República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación

Universidad Politécnica Territorial

José Antonio Anzoátequi

PNF EN INFORMÁTICA-EXTENSIÓN GUANTA

Informe - Repaso

Programación II

Profesor: Lennis Humer

Alumno: Sebastian Mota

09/11/2026

ÍNDICE

INTRODUCCIÓN.....

DESARROLLO..... 4

Tema 1: Fundamentos de la Programación Orientada a Objetos y UML 4

Clases e Instanciación de Objetos (Python y PHP) 4

Modelado Gráfico con Diagramas de Clases UML 4

Tema 2: Control de Visibilidad, Constructores y Herencia 4

Especificadores de Acceso (Public, Protected, Private) 4

Métodos Constructores e Inicialización 4

Herencia y Reutilización de Código 4

Tema 3: Clases Abstractas, Polimorfismo e Interfaces 5

Clases Abstractas y Plantillas Base 5

Polimorfismo y Sobrescritura de Métodos 5

Interfaces y Contratos de Métodos 5

Tema 4: Arquitectura Web y Diseño (MVC/MVT y HTML/CSS) 5

Patrones Arquitectónicos: MVC (Laravel) y MVT (Django) 6

Maquetación Web semántica y Motores de Plantillas (Blade y Jinja2) 6

CONCLUSIÓN 7

INTRODUCCIÓN

A lo largo de mi recorrido por la unidad curricular de Programación II, he podido comprender que la forma en que concebimos el software cambió radicalmente con la consolidación de la Programación Orientada a Objetos (POO). Ya no se trata solo de escribir código de forma lineal o desestructurada, sino de analizar y responder a problemas reales mediante clases, objetos y relaciones lógicas que permiten construir programas mantenibles, modulares y escalables.

En este informe consolidado presento una síntesis de los conocimientos que he ido adquiriendo clase a clase. El recorrido abarca desde la diferenciación entre clases e instancias y el modelado arquitectónico mediante el Lenguaje Unificado de Modelado (UML), pasando por los pilares fundamentales del encapsulamiento, los constructores y la herencia, hasta profundizar en mecanismos más avanzados como las clases abstractas, el polimorfismo y las interfaces. Finalmente, abordo la conexión directa entre la lógica orientada a objetos y el desarrollo web actual a través de patrones como MVC y MVT en frameworks como Laravel (PHP) y Django (Python).

DESARROLLO

Tema 1: Fundamentos de la Programación Orientada a Objetos y UML

Al iniciar la asignatura, el primer cambio importante en mi proceso de aprendizaje fue asimilar el paradigma de la Programación Orientada a Objetos. Aprendí a definir la clase como el molde o la plantilla teórica que especifica las propiedades (atributos) y los comportamientos (métodos), mientras que el objeto representa la instancia concreta de esa plantilla que se crea en la memoria del sistema.

Durante las sesiones prácticas comparé la sintaxis entre distintas tecnologías; por ejemplo, mientras que en Python la creación de un objeto se realiza mediante p1 = Persona(), en el lenguaje PHP la instanciación requiere la palabra clave new, como en $p1 = new Persona();.

Asimismo, comprendí la utilidad de los diagramas de clases UML como paso previo a la codificación. El modelado gráfico me ha permitido visualizar claramente la estructura de un software, definiendo de antemano la visibilidad de los atributos, los métodos disponibles y el tipo de relación (asociación, agregación, composición o herencia) entre los componentes del sistema.

Tema 2: Control de Visibilidad, Constructores y Herencia

Conforme avanzamos en los contenidos, profundicé en la aplicación del encapsulamiento para restringir el acceso directo a la información interna de un objeto mediante especificadores de acceso:

Público (public): Los miembros son accesibles desde cualquier parte del programa.

Protegido (protected): El acceso se limita a la propia clase y a sus clases derivadas.

Privado (private): Los datos son estrictamente accesibles dentro de la clase que los declara.

De igual forma, analicé la función del método constructor, el cual se ejecuta automáticamente al instanciar un objeto para garantizar que este inicie con sus valores o estado correcto. En Python esto se implementa mediante def __init__(self):, mientras que en PHP se utiliza public function __construct().

Respecto a la herencia, comprendí su valor como mecanismo principal para evitar la duplicación de código y establecer jerarquías lógicas. A través de ella, una clase hija extiende a una clase padre, heredando sus atributos y métodos. Un ejemplo claro de esto se observa en la relación entre una clase general y una derivada:

Python: class Alumno(Persona):

PHP: class Alumno extends Persona

Tema 3: Clases Abstractas, Polimorfismo e Interfaces

En una etapa más avanzada del curso abordé la abstracción de nivel superior. Aprendí que una clase abstracta actúa como una estructura base no instanciable directamente, cuyo propósito es definir una plantilla con métodos que sus clases hijas deben implementar obligatoriamente. En PHP esto se indica con la palabra clave abstract class, mientras que en Python se implementa a través del módulo from abc import ABC.

Por otro lado, entendí el polimorfismo como la capacidad que poseen distintos objetos pertenecientes a una misma jerarquía de responder de forma diferente ante un mismo método. Esto se logra mediante la sobrescritura de métodos (override) en las clases hijas, adaptando el comportamiento a la necesidad específica de cada una.

Por último, estudié las interfaces, las cuales funcionan como contratos formales que imponen una lista de métodos obligatorios que cualquier clase firmante debe desarrollar. Mientras que PHP cuenta con la sintaxis explícita interface, en Python esta estructura se trabaja habitualmente mediante prototipos y clases base abstractas (ABC / Protocol).

Tema 4: Arquitectura Web y Diseño (MVC/MVT y HTML/CSS)

Para integrar lo aprendido, analizamos el papel de la POO en el desarrollo web moderno, comprobando que los frameworks de software actuales estructuran su lógica siguiendo patrones de diseño arquitectónicos:

Patrón MVC (Modelo-Vista-Controlador - PHP/Laravel):

Modelo: Gestiona los datos y las reglas de negocio.

Vista: Presenta la interfaz visual al usuario.

Controlador: Procesa las solicitudes entrantes, consulta al modelo y retorna la vista correspondiente.

Patrón MVT (Modelo-Vista-Template - Python/Django):

Modelo: Define la estructura de datos mediante el ORM.

Vista: Contiene la lógica de control y procesamiento de respuestas.

Template: Maneja la maquetación visual mediante plantillas HTML.

En este apartado también relacioné la maquetación en HTML (definición de estructura y contenido semántico) y CSS (estilos y presentación visual) con el uso de motores de plantillas dinámicas como Jinja2 en Django (templates/) y Blade en Laravel (resources/views/), los cuales permiten conectar de forma fluida el trabajo del backend con la interfaz gráfica

CONCLUSIÓN

El recorrido práctico y teórico por los contenidos de la unidad curricular Programación II ha sido fundamental para consolidar mi criterio técnico sobre la construcción de soluciones de software estructuradas. Comprender los pilares de la POO —encapsulamiento, herencia, abstracción y polimorfismo— me ha demostrado que el diseño orientado a objetos no es un concepto puramente académico, sino una metodología indispensable para escribir código limpio, eficiente y reutilizable.

Adicionalmente, ver reflejados estos principios en herramientas de desarrollo real como Laravel y Django me ha permitido conectar la teoría de las clases y los métodos con la arquitectura de aplicaciones web completas. Esta experiencia refuerza mi capacidad lógica para planificar, diagramar y desarrollar proyectos informáticos con estándares profesionales.